مشروع تخرج — بكالوريوس علوم الحاسب

الإصدار 2.0 — نوفمبر 2026

ضاد

DHAD · ARABIC PROGRAMMING PLATFORM

منصة برمجة عربية متكاملة

لغة برمجة عربية بمحلل نحوي تزايدي، مفسر، مترجم إلى WebAssembly، خادم لغة (LSP)، محرر ويب بدعم RTL/BiDi أصيل، شارح أخطاء ذكي مقيَّد بالتوثيق، وبيئة تعلم تفاعلية — مع خطة تقييم تجريبي كاملة.

إعداد الطالب

_____

القسم / الكلية

علوم الحاسب — _____

المشرف الأكاديمي

_____

العام الأكاديمي

2026 / 2027 — فصلان دراسيان

المحتويات

جميع القرارات التصميمية الواردة في هذه الوثيقة مرشحة للمراجعة مع المشرف قبل الأسبوع الثالث من الفصل الأول.

1. الملخص التنفيذي 3

2. المشكلة والدافع 4

3. تحليل الوضع الحالي والفجوة البحثية 6

4. الرؤية والأهداف ومعايير النجاح 8

5. البنية المعمارية للمنصة 10

6. الطبقة الأولى: النواة اللغوية 11

تصميم اللغة، الكلمات المفتاحية، سياسة الأرقام وUnicode، أكواد الأخطاء، المفسر، مسار WASM

7. الطبقة الثانية: أدوات المطور 16

خادم LSP، إضافة VS Code، محرر الويب وطبقة BiDi، المنسّق، المدقق، ساحة التجريب

8. الطبقة الثالثة: الذكاء الاصطناعي المساعد 18

9. الطبقة الرابعة: بيئة التعلم 20

10. الجودة الهندسية والأمان وإتاحة الوصول 21

11. التقييم التجريبي ودراسة المستخدمين 23

12. المتطلبات الوظيفية وغير الوظيفية 25

13. خطة التنفيذ (فصلان دراسيان) والمعالم 27

14. ترتيب الأولويات (MoSCoW) ونطاق MVP 29

15. التقنيات وهيكلة المستودع 30

16. المخاطر وخطة التخفيف 32

17. التموضع أمام اللجنة والأسئلة المتوقعة 33

18. المراجع 34

الملحق أ — مسودة مواصفة اللغة (EBNF) 35

الملحق ب — فهرس أكواد الأخطاء الأولي 37

الملاحق ج، د، هـ — قالب ADR، استبيان SUS، قاموس المصطلحات 38

1 الملخص التنفيذي

يقترح هذا المشروع بناء **منصة برمجة عربية متكاملة** تحمل الاسم المقترح «ضاد» — نسبةً إلى الحرف الذي تُعرف به العربية بوصفها «لغة الضاد». المنصة ليست «لغة برمجة عربية أخرى»، بل نظام بيئي كامل يجعل الكتابة البرمجية بالعربية ممكنة عمليًا خارج نطاق الأمثلة التعليمية: من لحظة كتابة أول حرف في المحرر، مرورًا بالتحليل والتنفيذ والترجمة إلى WebAssembly، وصولًا إلى فهم الخطأ وتصحيحه بمساعدة ذكية مقيَّدة.

تتكون المنصة من **أربع طبقات هندسية أساسية** تدعمها ثلاث طبقات تمكينية:

1 · النواة اللغوية

قواعد Tree-sitter تزايدية، مفسر Tree-walking بلغة Rust، مولّد WebAssembly، مكتبة قياسية عربية مصغّرة، ونظام أكواد أخطاء موثَّق.

2 · أدوات المطور

خادم LSP (إكمال، تشخيص، تلميحات، انتقال للتعريف)، إضافة VS Code، منسّق ومدقق، ومحرر ويب بطبقة BiDi مخصصة.

3 · الذكاء الاصطناعي المساعد

شارح أخطاء عربي يعتمد على التوثيق الرسمي للغة (RAG) مع تلميحات تربوية متدرجة وحارس جودة — LLM كمساعد لا كمترجم.

4 · التقييم التجريبي

أداء WASM مقابل المفسر، زمن استجابة LSP، صحة BiDi، فهم رسائل الخطأ، وقابلية الاستخدام (SUS) في دراسة مستخدمين حقيقية.

والطبقات التمكينية: **بيئة تعلم** (دروس تفاعلية بمصحّح آلي في المتصفح)، **جودة هندسية** (تكامل مستمر، اختبارات fuzzing، golden، قياسات أداء آلية، سجل قرارات معمارية)، و**أمان وإتاحة وصول** (حدود موارد لتشغيل كود المستخدمين، دعم قارئات الشاشة، خط برمجي عربي).

4+3	2	28	9
طبقات معمارية	أهداف تنفيذ: مفسر + WASM	أسبوع عمل مخطط	مقاييس تقييم كمية

الفجوة التي يسدّها المشروع

لا توجد اليوم لغة عربية تجمع في آنٍ واحد: نواة بسيطة قابلة للتوسع ومتعددة الأهداف، وأدوات مطور حقيقية مبنية حول محلل تزايدي، ودعمًا هندسيًا أصيلًا لاتجاه الكتابة من اليمين إلى اليسار، ومساعدة ذكية للمبتدئين لا تمسّ حتمية المترجم. المشاريع السابقة إما بنت لغة تعليمية أحادية الهدف (Alif) أو لغة أنظمة معقدة (SultanC)؛ لم يبنِ أحد «النظام البيئي الوسيط».

الحد الأدنى القابل للتسليم (MVP): لغة تعمل + مفسر + REPL + أكواد أخطاء موثقة + خادم LSP أساسي + محرر ويب يعرض الكود العربي بشكل صحيح + تكامل مستمر. كل ما عدا ذلك مرتَّب بأولوية صريحة (MoSCoW) بحيث لا يهدد أي تأخير جوهرَ المشروع.

2 المشكلة والدافع

2.1 لماذا البرمجة بالعربية؟

تشير أدبيات تعليم البرمجة إلى أن العبء المعرفي على المبتدئ يتضاعف حين يتعلم مفهومًا جديدًا (المتغير، الحلقة، الدالة) بلغة أجنبية في الوقت نفسه. الطالب الناطق بالعربية في مراحله الأولى لا يصارع «منطق البرمجة» فقط بل يصارع المفردات الإنجليزية ورسائل الخطأ الإنجليزية معًا. لغة برمجة بالعربية لا تهدف إلى استبدال اللغات العالمية في سوق العمل، بل إلى **إزالة الحاجز الأول** وجعل المفهوم البرمجي يصل مباشرة.

2.2 لماذا فشلت المحاولات السابقة في الانتشار؟

المشكلة لم تكن يومًا في «ترجمة الكلمات المفتاحية». تحليل المشاريع القائمة يكشف أن ما ينقصها هو ما يحيط باللغة:

- **غياب الأدوات:** لا إكمال تلقائي، لا تشخيص فوري، لا تكامل مع المحررات الحديثة. المستخدم يكتب في محرر نصوص عادي ويشغّل من سطر الأوامر.
- **العرض المكسور للنص المختلط:** خوارزمية Unicode BiDi تُفسد عرض أي سطر يخلط العربية بالرموز والأرقام والمعرّفات اللاتينية، فتبدو الأقواس معكوسة والعمليات الحسابية مقلوبة. هذا وحده يكفي لإحباط المبتدئ.
- **رسائل خطأ غير مفهومة:** حتى حين تكون الرسالة بالعربية، تظل تقنية وجافة ولا تشرح للمبتدئ ماذا يفعل.
- **غياب التقييم:** معظم المشاريع تُقدَّم كعرض توضيحي بلا قياس لأثرها الفعلي على التعلم أو لأدائها الحقيقي.

2.3 الفرضية المركزية للمشروع

> «قيمة لغة البرمجة العربية لا تكمن في اللغة نفسها، بل في النظام البيئي الذي يحيط بها: أدوات تعمل، عرض صحيح، أخطاء مفهومة، وتعلم مُوجَّه. إذا بُنيت هذه الطبقات هندسيًا — لا كإضافات لاحقة — تصبح العربية صالحة للبرمجة الحقيقية.»

2.4 لمن هذه المنصة؟ (الشخصيات المستهدفة)

الشخصية	الوصف	ما تحتاجه من المنصة
ليلى — طالبة ثانوية	تتعلم البرمجة لأول مرة، إنجليزيتها متوسطة، تستخدم المتصفح فقط.	ساحة تجريب بلا تثبيت، دروس تفاعلية، أخطاء مشروحة ببساطة.
أحمد — معلم حاسب	يدرّس 30 طالبًا، يريد تمارين تُصحَّح آليًا ويتابع تقدم الطلاب.	مصحّح آلي، روابط مشاركة، مكتبة تمارين، تشغيل آمن للكود.

الشخصية	الوصف	ما تحتاجه من المنصة
سارة — طالبة جامعية	تعرف Python، تريد أداة حقيقية بمحرر حديث لمشروع صغير بالعربية.	إضافة VS Code، إكمال تلقائي، منسّق، أداء مقبول (WASM).
د. خالد — باحث	مهتم بتعليم البرمجة وبالذكاء الاصطناعي في التعليم.	بيانات تقييم قابلة لإعادة الإنتاج، كود مفتوح، منهجية موثقة.

3 تحليل الوضع الحالي والفجوة البحثية

3.1 اللغات العربية القائمة

اللغة	النهج التقني	نقطة القوة	القيد
ألف (Alif)	مبنية من الصفر بلغة C++، متعددة المنصات، كائنية التوجه	بيئة تطوير (Alif Studio)، مجتمع نشط، تتجنب عمدًا Lex/Yacc/LLVM	مترجم أحادي الهدف؛ لا WASM ولا LLVM؛ لا خادم LSP
SultanC	لغة أنظمة ذاتية الاستضافة، كلمات مفتاحية عربية وإنجليزية، ملكية وأنواع جبرية	توليد أصلي (ELF, Mach-O, PE,) WASM)، تمثيل وسيط موثَّق (MSIR)	معقدة (شبيهة بـ Rust)؛ غير موجهة للمبتدئين؛ لا بيئة ويب
زهراء (Zahra)	مترجم موحد (VM + QBE + LLVM) مع مرحلة تحقق ZIR	تعدد أهداف التجميع	نضج أقل، مجتمع صغير
عربي (arabi-lang)	موزعة عبر pip، تدعم التزامن وخوادم ويب	سهولة التثبيت	مبنية فوق Python؛ ليست مستقلة تقنيًا
العارب (Alarab)	مبنية بلغة C بهدف الاستقلال التقني	استقلالية	معلومات غير كافية عن نضج الأدوات

3.2 مصفوفة الفجوة

الجدول التالي يُظهر بوضوح موضع «ضاد» مقارنةً بأقرب مشروعين، وهو الشريحة التي تُعرض للجنة عند سؤال «ما الجديد؟».

الخاصية	Alif	SultanC	APL (بحث 2025)	ضاد (المقترح)
موجّهة للمبتدئين	✔	✘	✔	✔
محلل تزايدي (Tree-sitter)	✘	✘	✘	✔
خادم LSP + إضافة VS Code	✘	جزئي	✘	✔
هدف WebAssembly	✘	✔	✘	✔
تشغيل في المتصفح بلا تثبيت	✘	✘	✘	✔
طبقة BiDi هندسية في المحرر	✘	✘	✘	✔
أكواد أخطاء موثقة بالعربية	✘	✘	✘	✔
شرح أخطاء بـ LLM مقيَّد بالتوثيق	✘	✘	LLM كمترجم (غير حتمي)	✔ (LLM كمساعد فقط)

الخاصية	Alif	SultanC	APL (بحث 2025)	ضاد (المقترح)
دروس + مصحّح آلي	✘	✘	✘	✔
تقييم تجريبي مع مستخدمين	✘	✘	جزئي	✔ (SUS + اختبار فهم)

تمييز منهجي مهم عن APL

مشروع APL البحثي (2025) استخدم نماذج اللغة الكبيرة كمترجم شبه آلي يحوّل الكود العربي إلى Python، أي أن سلوك «اللغة» غير حتمي ويتبدل بتبدل النموذج. في «ضاد» لا يلمس النموذجُ اللغويُّ منطقَ الترجمة إطلاقًا: المحلل والمفسر والمترجم حتميون بالكامل، والنموذج يشرح فقط خطأً اكتشفه المترجم، ومخرجاته تمر عبر حارس جودة قبل العرض. هذا الفصل هو ما يجعل الطبقة الثالثة قابلة للتقييم علميًا.

3.3 الأعمال ذات الصلة خارج العالم العربي

- **Hedy** (هولندا): لغة تعليمية متدرجة تدعم كلمات مفتاحية بلغات متعددة منها العربية؛ تؤكد أن «الواجهة اللغوية» للمبتدئين مسألة بحثية جدية، لكنها تعمل فوق Python ولا تعالج مشكلة BiDi في المحرر ولا توفر LSP.
- **Rust diagnostics**: نموذج رسائل الخطأ المرقمة والموثقة (`E0382` مثلًا) الذي يُستلهم منه نظام أكواد «ضاد».
- **Python Tutor**: متتبّع التنفيذ المرئي الذي يُستلهم منه مكوّن التصوّر في بيئة التعلم.
- **أبحاث «Enhanced Compiler Error Messages»** (Becker وآخرون): تُظهر أن تحسين صياغة رسائل الخطأ يقلل تكرار الأخطاء عند المبتدئين — الأساس النظري لتجربة الطبقة الثالثة.

4 الرؤية والأهداف ومعايير النجاح

4.1 الرؤية

أن يتمكن أي ناطق بالعربية من فتح المتصفح وكتابة برنامج بلغته وتشغيله فورًا وفهم أخطائه بنفسه — بتجربة لا تقل جودةً عن أدوات اللغات العالمية.

4.2 الأهداف (قابلة للقياس)

الرمز	الهدف	مؤشر الإنجاز
هـ-1	تصميم لغة عربية صغيرة قابلة للتوسع بقواعد Tree-sitter رسمية ومفسر يعمل.	تحليل وتنفيذ 50 برنامجًا مرجعيًا بنجاح 100%
هـ-2	ترجمة مجموعة فرعية من اللغة إلى WebAssembly وتشغيلها في المتصفح.	تسريع ≤ 5× مقابل المفسر على خوارزميات رقمية
هـ-3	تقديم أدوات مطور حقيقية عبر LSP ومحرر ويب بعرض BiDi صحيح.	زمن استجابة p95 للإكمال < 100ms؛ اجتياز 100% من حالات BiDi
هـ-4	بناء شارح أخطاء عربي مقيَّد بالتوثيق وقياس أثره على الفهم.	تحسّن دال إحصائيًا في اختبار الفهم مقابل الرسالة الخام
هـ-5	إثبات قابلية الاستخدام مع مستخدمين حقيقيين.	درجة SUS ≥ 70 (فوق المتوسط العالمي 68)
هـ-6	تسليم المشروع كمنتج مفتوح المصدر موثق وقابل لإعادة الإنتاج.	مستودع عام، CI أخضر، إصدار v1.0 موسوم، توثيق ثنائي اللغة

4.3 ما ليس ضمن النطاق (صراحةً)

- بناء لغة أنظمة عالية الأداء منافسة لـ C/Rust — الهدف تعليمي وتطبيقي خفيف.
- مترجم LLVM كامل — يُذكر كعمل مستقبلي، وقد يُنفَّذ نموذج أولي إن سمح الوقت.
- الاستضافة الذاتية (كتابة مترجم ضاد بلغة ضاد) — عمل مستقبلي.
- استخدام LLM لتوليد أو ترجمة الكود كجزء من مسار التنفيذ — مرفوض تصميميًا.

4.4 معايير النجاح أمام اللجنة

يعمل فعلًا

عرض حي: كتابة برنامج عربي في المتصفح، تشغيله بالمفسر ثم بـ WASM، إحداث خطأ متعمد ورؤية شرحه.

مقاس كميًا

جداول ورسوم من بيانات حقيقية: أداء، زمن استجابة، نتائج دراسة المستخدمين مع تحليل إحصائي.

مهندَس باحتراف

مستودع منظم، اختبارات آلية، سجل قرارات، توثيق — لا «كود عرض» يعمل مرة واحدة.

5 البنية المعمارية للمنصة

تتبع المنصة معمارية طبقية: كل طبقة تعتمد على ما تحتها عبر واجهات واضحة، بحيث يمكن تسليم كل طبقة وتقييمها مستقلةً. **شجرة Tree-sitter** هي العمود الفقري المشترك: يستهلكها المفسر، والمترجم، وخادم LSP، والمنسّق، والمدقق — فلا يُكتب محلل مرتين.



الشكل 1 — المعمارية الطبقية لمنصة ضاد. شجرة التحليل الواحدة تُغذّي كل الأدوات.

5.1 تدفق البيانات الرئيسي

1. يكتب المستخدم الكود في المحرر (ويب أو VS Code). كل ضغطة مفتاح تُحدّث شجرة Tree-sitter تزايديًا.
2. يستهلك خادم LSP الشجرة لإنتاج التشخيصات والإكمال فورًا، دون انتظار التنفيذ.
3. عند التشغيل: المفسر ينفّذ الشجرة مباشرة (وضع التطوير)، أو المترجم يولّد وحدة WASM تُحمَّل في المتصفح (وضع الأداء).
4. عند الخطأ: يُصدر المترجم خطأً مرقمًا بموقعه؛ طبقة الذكاء الاصطناعي تأخذ الكود + صفحة التوثيق الرسمية لذلك الكود + مقتطف السياق وتولّد شرحًا متدرجًا يمر بحارس الجودة قبل العرض.
5. في وضع التعلم: المصحّح الآلي يشغّل كود الطالب في Web Worker معزول بحدود زمن وذاكرة على حالات اختبار خفية.

6 الطبقة الأولى: النواة اللغوية

6.1 فلسفة تصميم اللغة

- **صغيرة أولًا:** نواة من 12 كلمة مفتاحية تقريبًا تكفي لبرامج تعليمية ومتوسطة، وتوسّع لاحقًا دون كسر التوافق.
- **عربية طبيعية لا مترجمة حرفيًا:** اختيار الكلمات بناءً على ما يقرؤه العربي بسلاسة (`طالما` أوضح من «بينما» لحلقة while؛ `أرجع` أوضح من «عُد»).
- **صيغة أقواس معقوفة** `{}` لتفادي حساسية المسافات البادئة التي تربك المبتدئين وتربك العرض في بيئات RTL.
- **أنواع ديناميكية مع تحقق تدريجي:** البداية ديناميكية (أسهل للمبتدئ)، مع تلميحات أنواع اختيارية يستخدمها المترجم إلى WASM ويعرضها LSP.
- **كل خطأ له رقم وصفحة:** لا توجد رسالة خطأ بلا كود موثق.

6.2 الكلمات المفتاحية المقترحة

الكلمة	المقابل	الدور	مثال
`دع`	`let`	تعريف متغير قابل للتغيير	`دع العمر = ٢٥`
`ثابت`	`const`	تعريف ثابت	`ثابت ط = ٣٫١٤`
`إذا` / `وإلا إذا` / `وإلا`	`if / else if / else`	التفرع الشرطي	`إذا العمر >= ١٨ { ... } وإلا { ... }`
`طالما`	`while`	حلقة شرطية	`طالما س < ١٠ { س = س + ١ }`
`لكل` ... `في`	`for ... in`	حلقة تكرار على مدى أو قائمة	`لكل ع في ١..٥ { اطبع(ع) }`
`دالة`	`function`	تعريف دالة	`دالة جمع(أ، ب) { أرجع أ + ب }`
`أرجع`	`return`	إرجاع قيمة	`أرجع النتيجة`
`توقف` / `استمر`	`break / continue`	التحكم في الحلقات	`إذا س == ٥ { توقف }`
`صحيح` / `خطأ`	`true / false`	القيم المنطقية	`دع جاهز = صحيح`
`و` / `أو` / `ليس`	`and / or / not`	العمليات المنطقية	`إذا أ > ٠ و ليس ب { ... }`
`عدم`	`null / none`	غياب القيمة	`دع نتيجة = عدم`
`استورد`	`import`	استيراد وحدة	`استورد رياضيات`

الكلمة	المقابل	الدور	مثال
اختبر / تأكد	`test / assert`	اختبارات مدمجة في اللغة	`اختبر "الجمع" { تأكد(جمع(١،٢) == ٣) }`
صنف / هذا	`class / this`	(مرحلة لاحقة) كائنات بسيطة	`صنف نقطة { ... }`

الفاصلة العربية `،` والفاصلة اللاتينية `,` كلتاهما مقبولتان كفاصل معاملات؛ الفاصلة العشرية العربية `٫` مقبولة إلى جانب النقطة. يُوثَّق ذلك في مواصفة اللغة (الملحق أ).

6.3 برنامج نموذجي بلغة ضاد

```
استورد رياضيات

دالة مضروب(ن) {
  إذا ن <= ١ {
    أرجع ١
  }
  أرجع ن * مضروب(ن - ١)
}

دالة أولي؟(ع) {
  إذا ع < ٢ { أرجع خطأ }
  لكل ق في ٢..رياضيات.جذر(ع) + ١ {
    إذا ع % ق == ٠ { أرجع خطأ }
  }
  أرجع صحيح
}

دع الأعداد = [١، ٢، ٣، ٤، ٥، ٦، ٧]
لكل ع في الأعداد {
  إذا أولي؟(ع) {
    اطبع("${ع} عدد أولي ومضروبه ${مضروب(ع)}")
  }
}
```

لاحظ: علامة الاستفهام في اسم الدالة `أولي؟` مسموح بها كاصطلاح للدوال المنطقية (مستوحى من Ruby/Scheme) وتزيد قابلية القراءة بالعربية.

6.4 قرارات التصميم الحرجة (تُوثّق كـ ADR)

القرار 1 — الأرقام

يقبل المحلل اللغوي الأرقام العربية الشرقية `٠١٢٣٤٥٦٧٨٩` والغربية `0123456789` معًا، ويوحّدهما داخليًا إلى التمثيل الغربي. عرض الأرقام في المخرجات وفي المحرر خاضع لإعداد مستخدم (شرقي/غربي). السبب: المستخدمون في المشرق والمغرب العربي يختلفون في العادة، والتمثيل الداخلي الموحد يبسّط المترجم.

القرار 2 — سياسة Unicode للمعرّفات

- **تطبيع NFC** لكل معرّف قبل المقارنة والتخزين في جدول الرموز، حتى يتطابق المحرف المركّب والمفكك.
- **الحركات (التشكيل):** تُتجاهل في المعرّفات مع تحذير من المدقق (`عَدد` و `عدد` متغير واحد). تبقى الحركات ذات معنى داخل النصوص.
- **الهمزات والألف:** `اسم` و `إسم` معرّفان مختلفان (لا تطبيع دلالي)، لكن المدقق يحذر عند وجود معرّفين لا يختلفان إلا بالهمزة أو التاء المربوطة/الهاء.
- **المحارف المربكة (Confusables):** رفض خلط محارف من نصوص مختلفة تتشابه بصريًا في المعرّف الواحد (مثل الكاف العربية والكاف الفارسية)، استنادًا إلى UTS #39. هذه نقطة أصالة لم تعالجها اللغات العربية القائمة.
- **الأحرف الممدودة (الكشيدة ـ):** تُتجاهل في المعرّفات لأنها محرف زخرفي.

القرار 3 — Tree-sitter منذ البداية (مع ANTLR4 كخطة بديلة)

تُكتب القواعد في `grammar.js` وتُولَّد بـ Tree-sitter CLI محللًا بلغة C يُربط بـ Rust. الفائدة الحاسمة: المحلل نفسه يعمل في المفسر وفي خادم LSP وفي المحرر (عبر `web-tree-sitter` المترجم إلى WASM)، ويُنتج شجرة حتى عند وجود أخطاء (error recovery) وهو ما يحتاجه التشخيص أثناء الكتابة. إن ظهرت عقبة في دعم Unicode داخل القواعد يُبنى نموذج أولي بـ ANTLR4 الذي يدعم Unicode كاملًا.

القرار 4 — Rust لغة تنفيذ

الأداء، سلامة الذاكرة، نضج منظومة الأدوات ذات الصلة (`tree-sitter`, `tower-lsp`, `wasm-encoder`, `wasmtime`)، وإمكانية ترجمة المفسر نفسه إلى WASM ليعمل في المتصفح بلا خادم. SultanC وZahra أثبتتا جدوى Rust لهذا النوع من المشاريع.

6.5 نظام أكواد الأخطاء الموثَّق

كل خطأ يحمل كودًا بصيغة `ضاد-XXXX` ضمن نطاقات: 0001–0999 أخطاء تحليل نحوي، 1000–1999 أخطاء دلالية وأنواع، 2000–2999 أخطاء وقت تشغيل، 3000–3999 تحذيرات المدقق. ولكل كود صفحة توثيق عربية (السبب، أمثلة خاطئة وصحيحة، أخطاء مشابهة). مثال مخرجات المترجم:

```
خطأ[ضاد-1003]: لا يمكن جمع نص مع عدد
  ← الملف: مثال.ضاد — السطر ٣، العمود ١٨
     |
   ٣ |   دع الرسالة = "العمر: " + ٢٥
     |                ^^^^^^^^^^^^ — عدد صحيح
     |                نص
     |
  = مساعدة: حوّل العدد إلى نص بـ نص(٢٥)، أو استخدم دمج النصوص: "العمر: ${٢٥}"
  = لمزيد من التفاصيل: ضاد شرح 1003
```

لماذا هذا الاستثمار يعود بفائدة مضاعفة؟

صفحات التوثيق هذه هي نفسها «مصدر الحقيقة» الذي يُحقن في برومبت شارح الأخطاء (الطبقة الثالثة). فبدل أن يعتمد النموذج اللغوي على معرفته العامة، يُطلب منه إعادة صياغة الصفحة الرسمية فقط — مما يحوّل مشكلة الهلوسة من «أمل» إلى «قيد قابل للقياس».

6.6 المفسر و REPL

مفسر Tree-walking يمشي على شجرة Tree-sitter مباشرة (مع مرحلة تحويل خفيفة إلى AST مبسّط لتسريع التنفيذ وتجنب إعادة الاستعلام عن العقد). يدعم: الأنواع الأساسية (عدد صحيح 64-بت، عدد عشري، نص، منطقي، عدم)، القوائم والقواميس، الدوال من الدرجة الأولى والإغلاقات، النطاقات اللغوية، والاستثناءات الأساسية. يُرفق به **REPL** بأمر `ضاد` بلا معاملات، بسجل أوامر وإكمال بسيط، ونسخة ويب منه في الساحة.

6.7 المكتبة القياسية العربية المصغّرة

الوحدة	أهم الدوال
أساسيات	`اطبع` ، `اقرأ` ، `نص` ، `عدد` ، `طول` ، `نوع` — متاحة بلا استيراد
رياضيات	`جذر` ، `قوة` ، `مطلق` ، `تقريب` ، `أكبر` ، `أصغر` ، الثابتان `ط` و `هـ`
نصوص	`قسّم` ، `اربط` ، `استبدل` ، `يبدأ_بـ` ، `احذف_التشكيل` ، `طبّع`
قوائم	`أضف` ، `رتّب` ، `اعكس` ، `صفِّ` ، `حوّل` ، `اجمع`
عشوائي	`عدد_عشوائي` ، `اختر` ، `اخلط`
وقت	`الآن` ، `انتظر` ، `قِس` (لقياس الزمن في التجارب)

تُكتب أجزاء من المكتبة (مثل `قوائم`) بلغة ضاد نفسها (dogfooding) — دليل عملي على نضج اللغة يُعرض للجنة.

6.8 مسار الترجمة إلى WebAssembly

المرحلة الأولى تستهدف **مجموعة فرعية رقمية** من اللغة (أعداد، منطق، شروط، حلقات، دوال، استدعاء ذاتي) تكفي لخوارزميات القياس. يُنتج المترجم وحدة WASM ثنائية مباشرة عبر صندوق `wasm-encoder` في Rust (بديلًا عن توليد نص WAT ثم تمريره لـ `wat2wasm`؛ يبقى إخراج WAT متاحًا بخيار `--wat` لأغراض التوثيق والتصحيح). النصوص والقوائم تُضاف في مرحلة ثانية عبر ذاكرة خطية مُدارة ودوال وقت تشغيل صغيرة.



الشكل 2 — خط الأنابيب الموحد: تحليل واحد، مساران للتنفيذ (تفسير مباشر أو وحدة WebAssembly).

لماذا WebAssembly أولًا؟ (أ) تشغيل في المتصفح بلا تثبيت — وهو شرط وصول الشخصيات المستهدفة؛ (ب) بيئة تنفيذ معزولة وآمنة بطبيعتها، تناسب تشغيل كود الطلاب؛ (ج) محمولية كاملة دون بناء backend أصلي معقد؛ (د) يتيح مقارنة أداء عادلة وقابلة للقياس مع المفسر. **LLVM** (عبر `inkwell`) أو **QBE** يبقيان عملًا مستقبليًا موثقًا.

7 الطبقة الثانية: أدوات المطور

7.1 خادم اللغة (LSP)

يُبنى بصندوق `tower-lsp` ويعمل فوق شجرة Tree-sitter نفسها، فلا يحتاج إلى تشغيل البرنامج لإنتاج معلوماته. الخدمات المخططة بالترتيب:

الخدمة	الوصف	الأولوية
التشخيص (Diagnostics)	أخطاء التحليل والدلالة فورًا أثناء الكتابة، بكود ضاد-XXXX ورسالة عربية ورابط للتوثيق	Must
الإكمال التلقائي	الكلمات المفتاحية، المتغيرات والدوال في النطاق، دوال المكتبة القياسية مع توقيعها بالعربية	Must
التلميح (Hover)	نوع المتغير المستنتج، توقيع الدالة، وتعليق التوثيق إن وُجد	Must
الانتقال إلى التعريف / المراجع	عبر جدول الرموز المبني من الشجرة	Should
إعادة التسمية	إعادة تسمية آمنة لمعرّف في كل مواضعه	Should
إجراءات الإصلاح السريع (Code Actions)	إصلاحات مقترحة من فهرس الأخطاء (مثل إدراج `نص(...)`)	Could
الترميز الدلالي (Semantic Tokens)	تلوين دقيق حسب الدلالة لا حسب النمط النصي فقط	Could

7.2 إضافة VS Code

تُنشر على Marketplace وتتضمن: عميل LSP، تلوين صياغة بملف استعلامات Tree-sitter (`highlights.scm`) وقواعد TextMate كخيار احتياطي، أيقونة لملفات `.ضاد`، مقتطفات (snippets) عربية للبُنى الشائعة، وأمر «تشغيل» يفتح المخرجات في لوحة طرفية بوضع RTL. نشر الإضافة علنًا يجعل المشروع «منتجًا منشورًا» لا مجرد تسليم أكاديمي.

7.3 محرر الويب وطبقة BiDi — التحدي التقني الأصعب

المشكلة بدقة

تحدد خوارزمية Unicode BiDi (UAX #9) اتجاه الفقرة من **أول محرف قوي الاتجاه**. السطر `result = مجموع + ٥` يبدأ بمعرّف لاتيني فتُعامل الفقرة كلها كـ LTR، بينما السطر العربي المحض يُعامل كـ RTL. والأسوأ: الرموز المحايدة (`=` ، `+` ، الأقواس) تأخذ اتجاهها من جيرانها، فتظهر الأقواس «مقلوبة» بصريًا وتتبادل أطراف العملية الحسابية مواضعها. في النثر هذا إزعاج؛ في الكود هذا يغيّر المعنى المقروء.

الحل الهندسي المقترح

1. **الاتجاه الأساسي للمحرر RTL دائمًا**، لا اعتمادًا على أول حرف قوي — فاللغة عربية بالتصميم. (CodeMirror 6 يدعم `direction: rtl` على مستوى المحرر وحركة المؤشر البصرية في النص المختلط.)
2. **عزل اتجاهي على مستوى الرموز (Token-level isolation):** لأن المحرر يملك شجرة Tree-sitter، يمكنه معرفة نوع كل رمز، فيغلّف المعرّفات اللاتينية والأرقام والنصوص المقتبسة بعناصر `bdi`/خصائص `unicode-bidi: isolate` عبر Decorations، بحيث لا «يتسرب» اتجاهها إلى الرموز المجاورة. هذا هو الفرق الجوهري بين «محرر نص عربي» و«محرر كود عربي».
3. **معاملة المشغّلات كمحارف قوية RTL ضمن سياق الكود:** ضبط عرض `{}[]()` والمشغّلات بحيث تتبع الاتجاه المنطقي للكود لا اتجاه الجار، مع عكس أشكال الأقواس المرآتية تلقائيًا (Bidi Mirroring).
4. **أرقام متسقة:** الأرقام الغربية داخل سياق RTL تُعزل كوحدة LTR واحدة حتى لا ينعكس ترتيب خاناتها؛ والأرقام الشرقية تُعرض طبيعيًا.
5. **زر تبديل يدوي** للاتجاه الأساسي ووضع «إظهار حدود العزل» لأغراض التصحيح والتعليم.

مجموعة اختبار BiDi (جزء من التقييم)

تُبنى مجموعة من 40+ حالة اختبار بصرية (snapshot tests عبر Playwright) تغطي: نص عربي محض، معرّف لاتيني في بداية السطر، نص مقتبس يحوي إنجليزية، أرقام شرقية وغربية في تعبير واحد، أقواس متداخلة، تعليقات مختلطة، وسلاسل طويلة تلتف على أسطر. لكل حالة «عرض متوقع» مرجعي تُقارن به اللقطة بكسلًا بكسل.

7.4 المنسّق والمدقق

ضاد نسّق (Formatter)

أسلوب تنسيق قياسي واحد لا يقبل الخيارات (على نهج gofmt)، مبني على شجرة Tree-sitter: مسافات بادئة، فواصل، التفاف الأسطر، توحيد الفاصلة العربية. يعمل من سطر الأوامر، ومن LSP (تنسيق عند الحفظ)، ومن محرر الويب.

ضاد دقّق (Linter)

قواعد تحذير (نطاق 3000–3999): متغير غير مستخدم، تظليل متغير، معرّفات متشابهة بصريًا (همزة/تاء مربوطة)، مقارنة ثابتة دائمًا، حلقة بلا تغيير لشرطها. تُعرض عبر LSP كتحذيرات ويمكن تعطيل كل قاعدة بتعليق.

7.5 ساحة التجريب (Playground)

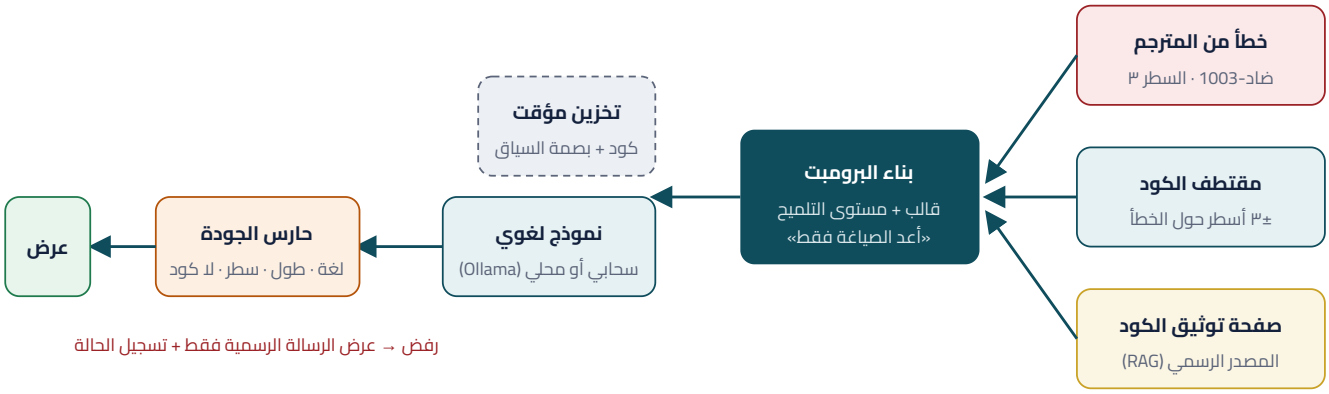
صفحة ويب واحدة بلا خادم تنفيذ: المحرر + المفسر المترجم إلى WASM يعمل في Web Worker + لوحة مخرجات RTL + REPL. زر «مشاركة» يولّد رابطًا دائمًا بالكود مضغوطًا في عنوان الصفحة (`lz-string`)، وزر «رمز QR» ليفتحه أعضاء اللجنة على هواتفهم أثناء المناقشة. الساحة هي بوابة الدخول لكل الشخصيات المستهدفة وهي واجهة دراسة المستخدمين.

8 الطبقة الثالثة: الذكاء الاصطناعي المساعد

المبدأ الحاكم

النموذج اللغوي الكبير **لا يترجم ولا ينفّذ ولا يقرر**. المترجم الحتمي يكتشف الخطأ ويحدد كوده وموقعه؛ النموذج يشرح فقط، مقيَّدًا بالتوثيق الرسمي، ومخرجاته تمر بحارس جودة، ولا يُنفَّذ أي كود يقترحه تلقائيًا. هذا الفصل يحافظ على حتمية المنصة ويجعل الطبقة قابلة للتقييم.

8.1 شارح الأخطاء الذكي — المعمارية



الشكل 3 — خط أنابيب شارح الأخطاء: مدخلات حتمية، نموذج مقيَّد، حارس قبل العرض.

8.2 التلميحات التربوية المتدرجة (Progressive Hints)

بدل تقديم الحل كاملًا — وهو ما يحوّل الأداة إلى «حلّال واجبات» — يقدَّم الشرح على أربعة مستويات يطلبها الطالب تباعًا، وفق مبدأ السقالات التعليمية (Scaffolding):

المستوى	المحتوى	مثال لخطأ ضاد-1003
1 — ماذا؟	إعادة صياغة مبسطة للرسالة	«حاولت أن تجمع كلمات مع رقم، واللغة لا تعرف كيف تجمعهما.»
2 — أين ولماذا؟	تحديد السطر والمفهوم	«في السطر ٣: "العمر: " نصٌّ و ٢٥ عدد. الجمع بـ + يعمل بين عددين أو بين نصين فقط.»
3 — مثال مشابه	حالة مماثلة محلولة بدون لمس كود الطالب	«مثل: `"النتيجة: " + نص(٧)` أو `"النتيجة: ${٧}"`»
4 — الإصلاح	الاقتراح على كود الطالب (يُعرض ولا يُطبَّق تلقائيًا)	«غيّر السطر إلى: `دع الرسالة = "العمر: ${٢٥}"`»

يُسجَّل المستوى الذي يتوقف عنده الطالب كمؤشر تعلم في دراسة المستخدمين (كلما حُلَّ الخطأ بمستوى أدنى، دلّ ذلك على فهم أعمق).

8.3 حارس الجودة (Guardrails)

- اللغة عربية (كشف لغوي بسيط) وبطول لا يتجاوز حدًا لكل مستوى.
- الإشارة إلى رقم السطر الصحيح نفسه الذي حدده المترجم.
- لا تتضمن كتلة كود في المستويات 1–3؛ وفي المستوى 4 يُعرض الكود كاقتراح فقط ويُتحقق من أنه يُحلَّل نحويًا بنجاح قبل العرض.
- لا تتعارض مع صفحة التوثيق (فحص تداخل مفردات بسيط + مراجعة بشرية بالعينة في التقييم).
- عند الرفض: تُعرض رسالة المترجم الرسمية وحدها، ويُسجَّل معدل الرفض كمقياس.

8.4 الاستقلال عن المزوّد ونموذج محلي احتياطي

تُعزل واجهة النموذج خلف طبقة تجريد واحدة بحيث يمكن التبديل بين واجهة سحابية (Claude أو GPT) ونموذج مفتوح يعمل محليًا عبر Ollama (ويُفضَّل نموذج عربي مثل Jais أو AceGPT). الفوائد: العرض الحي لا يرتهن بالإنترنت، الخصوصية في بيئات المدارس، **ومقارنة تجريبية** سحابي/محلي في دقة الشرح وسلامة العربية وزمن الاستجابة تُثري فصل التقييم.

8.5 قالب البرومبت (مبسّط)

```
أنت مساعد تعليمي يشرح أخطاء البرمجة لمبتدئ ناطق بالعربية.
لديك: (١) رسالة الخطأ الرسمية ورقمها، (٢) صفحة التوثيق الرسمية لهذا الخطأ، (٣) مقتطف الكود.
قواعد صارمة:
- أعد صياغة ما في صفحة التوثيق بلغة أبسط. لا تضف معلومات من خارجها.
- اذكر السطر {السطر} فقط. لا تذكر أسطرًا أخرى.
- المستوى المطلوب: {المستوى}. التزم بحدوده: {وصف المستوى}.
- أجب بالعربية الفصحى المبسطة في {الحد الأقصى} كلمة كحد أقصى. لا تكتب كودًا إلا في المستوى ٤.
```

8.6 مولّد الكود (اختياري، للمراجعة فقط)

واجهة تأخذ وصفًا عربيًا («اكتب دالة تحسب المضروب») وتعرض كودًا مولَّدًا في المحرر بحالة «مسودة» تتطلب نقرة قبول، ولا تُنفَّذ أبدًا تلقائيًا. تُقدَّم كعرض واعٍ لحدود الاعتماد على النماذج، لا كميزة أساسية.

9 الطبقة الرابعة: بيئة التعلم

تحوّل هذه الطبقة المنصة من «أداة» إلى «بيئة تعلم» وتوفر السياق الطبيعي لدراسة المستخدمين. تكلفتها منخفضة نسبيًا لأنها تُبنى فوق الساحة والمفسر والمصحّح.

9.1 مسار دروس تفاعلي داخل الساحة

#	الدرس	المفهوم	التمرين المصحَّح آليًا
1	مرحبًا يا عالم	الطباعة، النصوص، تشغيل البرنامج	اطبع اسمك في ثلاثة أسطر
2	الصناديق (المتغيرات)	دع، الأنواع الأساسية، الدمج {}$	احسب عمرك بعد ١٠ سنوات
3	القرارات	إذا / وإلا، المقارنات، و/أو	تصنيف درجة إلى تقدير
4	التكرار	طالما، لكل..في، توقف	جدول الضرب لعدد مُدخل
5	القوائم	إنشاء، فهرسة، إضافة، طول	أكبر عدد في قائمة
6	الدوال	تعريف، معاملات، أرجع	دالة تختبر الأولية
7	الأخطاء صديقتك	قراءة رسائل ضاد-XXXX، استخدام التلميحات	إصلاح برنامج فيه ٥ أخطاء
8	النصوص بعمق	وحدة نصوص، التشكيل والتطبيع	عدّ الكلمات في فقرة
9	مشروع صغير	دمج المفاهيم	لعبة تخمين العدد
10	تحت الغطاء	المفسر مقابل WASM، لماذا أسرع؟	قياس زمن خوارزمية بالطريقتين

9.2 المصحّح الآلي (Autograder) في المتصفح

يشغّل كود الطالب داخل Web Worker معزول على حالات اختبار خفية (مدخلات/مخرجات متوقعة) بمهلة زمنية وسقف ذاكرة، ويعرض نتيجة لكل حالة مع تلميح متدرج عند الفشل. لا يحتاج خادمًا — ما يجعله قابلًا للنشر كصفحة ثابتة واستخدامه في المدارس دون بنية تحتية.

9.3 متتبّع التنفيذ المرئي (Execution Visualizer)

وضع «خطوة بخطوة» فوق المفسر يعرض السطر الحالي، قيم المتغيرات في كل نطاق، ومكدس الاستدعاءات — على غرار Python Tutor لكن بالعربية وبعرض RTL. بديل أخف من منقّح كامل عبر بروتوكول DAP وأثره التعليمي أكبر؛ ويبقى DAP عملًا مستقبليًا يُبنى على نفس الخطافات (hooks) في المفسر.

9.4 التوثيق المولَّد والتعليقات التوثيقية

تعليقات بصيغة `## يحسب مضروب العدد ن` فوق الدوال يستخرجها أمر `ضاد وثّق` إلى موقع توثيق ثابت، ويعرضها LSP في التلميحات، وتُغذّى بها قاعدة معرفة الشارح.

10 الجودة الهندسية والأمان وإتاحة الوصول

10.1 الجودة الهندسية

الممارسة	التطبيق في المشروع	ما تثبته للجنة
التكامل المستمر (GitHub Actions)	على كل دفعة: بناء Rust، اختبارات الوحدة، مجموعة البرامج المرجعية (corpus)، اختبارات golden لشجرة التحليل، اختبارات BiDi البصرية، فحص التنسيق	أي تغيير يكسر اللغة يُكشف قبل الدمج
اختبارات golden/snapshot	لكل برنامج مرجعي: الشجرة المتوقعة (`tree-sitter test`) والمخرجات المتوقعة والأخطاء المتوقعة	استقرار السلوك عبر الإصدارات
Fuzzing	تغذية المحلل والمفسر بمدخلات مشوّهة (`cargo-fuzz` + أداة fuzz في Tree-sitter) للتحقق من عدم الانهيار وإصدار خطأ نظيف دائمًا	متانة نادرة في مشاريع الطلاب
قياسات أداء آلية	`criterion` لقياس المفسر وWASM على خوارزميات ثابتة؛ النتائج تُحفظ كملفات JSON وتُرسم في التقرير	أرقام التقييم قابلة لإعادة الإنتاج
سجل القرارات المعمارية (ADR)	ملف قصير لكل قرار كبير (الأرقام، WASM، Rust، Unicode أولًا، تصميم الكلمات)	نضج هندسي ويكتب فصل المنهجية
الإصدارات والترخيص	SemVer، رخصة MIT أو Apache-2.0، README ثنائي اللغة، دليل مساهمة	مشروع حي قابل للاستشهاد به

10.2 الأمان وقيود التشغيل

- **عزل تنفيذ كود المستخدم:** في المتصفح داخل Web Worker منفصل عن واجهة المحرر، وفي WASM داخل بيئة التشغيل المعزولة بطبيعتها.
- **حدود الموارد:** سقف ذاكرة لوحدة WASM، ومهلة زمنية تُنهي الـ Worker عند الحلقات اللانهائية، و«عدّاد وقود» (fuel) في المفسر يوقف التنفيذ بعد عدد من الخطوات في وضع التصحيح الآلي.
- **مخرجات النموذج اللغوي:** لا تُنفَّذ تلقائيًا أبدًا؛ تُعرض كنص وتُعقَّم قبل الإدراج في الصفحة.
- **الخصوصية:** خيار النموذج المحلي يعني أن كود الطالب لا يغادر الجهاز؛ ولا تُرسل بيانات تعريفية في طلبات الواجهة السحابية.
- **روابط المشاركة:** الكود يُضمَّن في الرابط نفسه؛ لا تخزين على خادم في النسخة الأولى (لا بيانات شخصية).

10.3 إتاحة الوصول وتجربة الاستخدام

- **خط برمجي عربي:** استخدام خط أحادي المسافة يدعم العربية (مثل Kawkab Mono) مع محاذاة سليمة للأرقام والرموز.
- **تبديل عرض الأرقام** شرقي/غربي وحجم الخط ووضع التباين العالي والوضع الليلي.

- **قارئات الشاشة:** تسميات ARIA للمحرر ولوحة المخرجات، وإعلان التشخيصات نصيًا، والتنقل الكامل بلوحة المفاتيح.
- **قابلية الاستخدام المقاسة:** استبيان SUS المعياري (الملحق د) بعد كل جلسة في دراسة المستخدمين.

11 التقييم التجريبي ودراسة المستخدمين

الهدف من هذا الفصل إثبات أن المنصة «تعمل» بمقاييس قابلة للقياس وإعادة الإنتاج، لا بعرض توضيحي. يتكون التقييم من شقين: **قياسات تقنية آلية** و**دراسة مستخدمين**.

11.1 المقاييس التقنية

المقياس	الطريقة	الهدف
أداء WASM مقابل المفسر	5 خوارزميات (فيبوناتشي تكراري، فرز فقاعي، غربال إراتوستينس، ضرب مصفوفات، بحث ثنائي) × 3 أحجام × 10 تكرارات؛ قياس الوسيط والانحراف	تسريع ≥ 5× على الحمل الرقمي
زمن استجابة LSP	p50/p95 للإكمال والتشخيص على ملفات بأحجام 50/500/5000 سطر	p95 < 100ms للإكمال
تغطية القواعد	مجموعة 50+ برنامجًا مرجعيًا من بسيط إلى متوسط؛ نسبة التحليل والتنفيذ الصحيحين	100%
صحة BiDi	40+ حالة بصرية؛ مقارنة لقطات Playwright بالمرجع	100% تطابق
متانة المحلل	مليون مدخل fuzzing؛ نسبة الانهيارات	0 انهيار
دقة الإكمال	مجموعة 100 موضع إكمال مُعلَّمة يدويًا؛ Precision@5	≥ 0.9
جودة شرح النموذج	100 خطأ مرجعي × نموذجين (سحابي/محلي)؛ تقييم مقيّمَين مستقلّين بمقياس 1–5 (الدقة، البساطة، سلامة العربية) مع معامل اتفاق Cohen's κ	متوسط ≥ 4 و κ ≥ 0.6
معدل رفض الحارس	نسبة المخرجات المرفوضة وأسبابها	يُبلَّغ عنه (مؤشر وعي بالحدود)

11.2 دراسة المستخدمين

المشاركون	24–30 طالبًا مبتدئًا (ثانوي/سنة أولى) بلا خبرة برمجية سابقة؛ موافقة مستنيرة؛ موافقة لجنة الأخلاقيات إن تطلب الأمر
التصميم	بين المجموعات (Between-subjects): المجموعة أ ترى رسائل الخطأ الرسمية فقط؛ المجموعة ب ترى الرسائل + الشرح المتدرج. توزيع عشوائي متوازن
المهمة	الدروس 1–4 ثم تمرين «إصلاح برنامج فيه 5 أخطاء» خلال 45 دقيقة في الساحة
القياسات	اختبار فهم قبلي/بعدي (10 أسئلة)، عدد الأخطاء المُصلَحة، زمن الإصلاح، أعلى مستوى تلميح مستخدم، درجة SUS، مقابلة قصيرة
التحليل	اختبار t أو Mann–Whitney للفروق بين المجموعتين؛ حجم الأثر (Cohen's d)؛ تحليل موضوعي (thematic) للمقابلات

الفرضيات	ف1: المجموعة ب تصلح أخطاءً أكثر في وقت أقل. ف2: تحسّن الفهم البعدي أعلى في ب. ف3: SUS ≥ 70 للمنصة ككل. ف4: لا فرق دال في الزمن بين النموذج السحابي والمحلي يبرر الاعتماد على السحابي وحده
الخطة البديلة	إن تعذر العدد: 8–12 مشاركًا بتصميم داخل المجموعات (Within-subjects) مع تحليل نوعي معمّق — تظل نتائج SUS ذات قيمة أولية

11.3 المقارنة المباشرة مع Alif

تُكتب خمسة برامج متطابقة بلغتي ضاد وAlif وتُقارن: عدد الأسطر، وضوح رسائل الخطأ (تقييم مقيّمين)، إمكانية التشغيل في المتصفح، وتوفر الإكمال التلقائي. هدفها التموضع التنافسي المدعوم بالدليل لا «الانتصار».

12 المتطلبات الوظيفية وغير الوظيفية

12.1 المتطلبات الوظيفية

الرمز	المتطلب	الأولوية
و-01	يحلل النظام ملفات .ضاد وفق قواعد Tree-sitter وينتج شجرة حتى عند وجود أخطاء	Must
و-02	ينفّذ المفسر المتغيرات والأنواع الأساسية والشروط والحلقات والدوال والقوائم والقواميس	Must
و-03	يقبل المحلل الأرقام الشرقية والغربية ويوحّدها داخليًا	Must
و-04	يطبّق المحلل سياسة Unicode للمعرّفات (NFC، تجاهل التشكيل، رفض المحارف المربكة)	Must
و-05	يُصدر النظام كل خطأ بكود ضاد-XXXX وموقع دقيق ورسالة عربية ورابط توثيق	Must
و-06	يوفر REPL تفاعليًا في الطرفية وفي الساحة	Must
و-07	يوفر خادم LSP التشخيص والإكمال والتلميح	Must
و-08	يعرض محرر الويب الكود العربي المختلط بشكل صحيح باتجاه أساسي RTL وعزل رموز	Must
و-09	يترجم النظام المجموعة الفرعية الرقمية إلى وحدة WASM تعمل في المتصفح	Should
و-10	يولّد الشارح شرحًا عربيًا متدرجًا مقيَّدًا بصفحة التوثيق ويمرره بحارس جودة	Should
و-11	تولّد الساحة روابط مشاركة دائمة ورمز QR	Should
و-12	يعيد المنسّق تنسيق الملفات بأسلوب قياسي	Should
و-13	تقدّم بيئة التعلم 10 دروس بمصحّح آلي يعمل في المتصفح	Should
و-14	يعرض المتتبّع المرئي التنفيذ خطوة بخطوة	Could
و-15	يدعم الشارح التبديل بين نموذج سحابي ومحلي	Could
و-16	يوفر المدقق قواعد التحذير الأساسية	Could
و-17	يدعم النظام استيراد حزم من مستودع بسيط	Could
و-18	مولّد كود من وصف نصي للمراجعة فقط	Could

12.2 المتطلبات غير الوظيفية

الرمز	المتطلب	المعيار
غ-01	الأداء	إكمال LSP p95 < 100ms؛ المفسر ×WASM ≥ 5

الرمز	المتطلب	المعيار
غ-02	المتانة	0 انهيار تحت fuzzing؛ خطأ نظيف لكل مدخل مشوّه
غ-03	الأمان	عزل تنفيذ كود المستخدم؛ حدود زمن وذاكرة؛ لا تنفيذ تلقائي لمخرجات النموذج
غ-04	الحتمية	نفس المدخل → نفس الشجرة والمخرجات والأخطاء دائمًا؛ النموذج اللغوي خارج مسار التنفيذ
غ-05	قابلية الاستخدام	SUS ≥ 70؛ تشغيل في المتصفح بلا تثبيت
غ-06	إتاحة الوصول	تنقل كامل بلوحة المفاتيح؛ ARIA؛ تباين وفق WCAG AA
غ-07	قابلية الصيانة	تغطية اختبارات ≥ 80% للنواة؛ ADR لكل قرار كبير؛ CI أخضر
غ-08	المحمولية	الأدوات تعمل على Linux وmacOS وWindows؛ الساحة على المتصفحات الحديثة والجوال
غ-09	قابلية إعادة الإنتاج	سكربت واحد يعيد توليد كل أرقام التقييم من المستودع

13 خطة التنفيذ (فصلان دراسيان) والمعالم

1م	2م	3م	4م	5م	6م
الأسبوع 2 وثيقة التصميم + ADRs	الأسبوع 5 Grammar + Corpus	الأسبوع 9 مفسر + REPL + أخطاء	الأسبوع 14 WASM + LSP + VS Code	الأسبوع 20 محرر BiDi + ساحة + شارح	الأسبوع 28 تقييم + تقرير + عرض

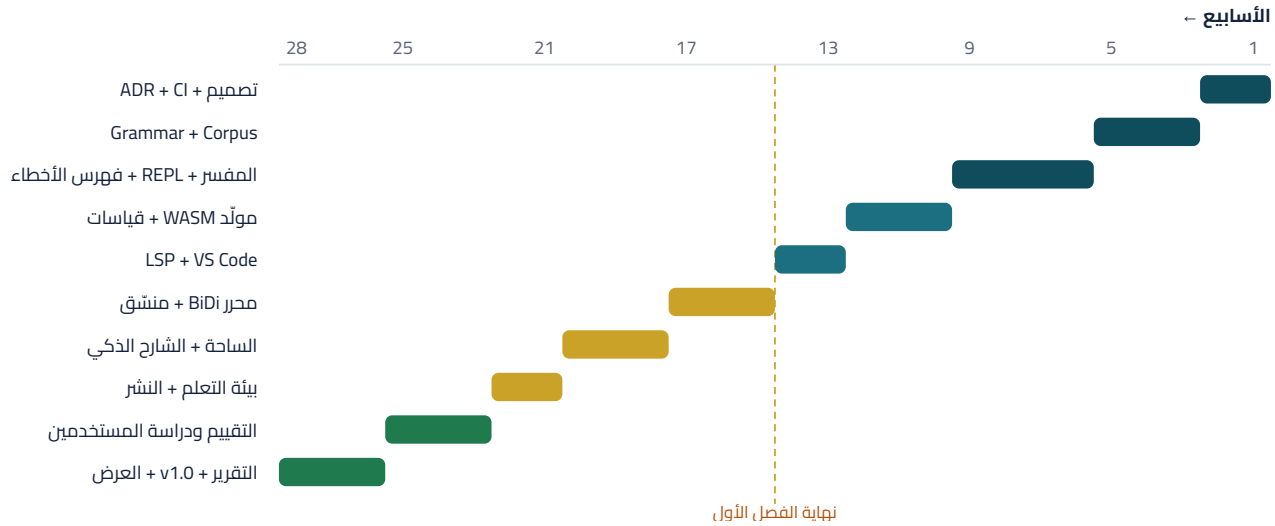
13.1 الفصل الأول — النواة والأدوات الأساسية

الأسابيع	المهام	المخرج القابل للتحقق
2–1	مراجعة الأدبيات؛ مقارنة Tree-sitter/ANTLR4 بنموذج أولي صغير؛ تثبيت الكلمات المفتاحية وسياسة الأرقام وUnicode؛ إعداد المستودع وCI وقالب ADR	وثيقة تصميم اللغة v1 + 5 ملفات ADR + CI يعمل
5–3	كتابة `grammar.js`؛ مجموعة اختبارات corpus (30 برنامجًا)؛ ملف تلوين `highlights.scm`؛ ربط المحلل بـ Rust	المحلل يجتاز 100% من corpus؛ تلوين يعمل في VS Code بلا LSP
9–6	المفسر: القيم والبيئات والدوال والإغلاقات؛ فهرس الأخطاء (أول 30 كودًا بصفحاتها)؛ REPL؛ نواة المكتبة القياسية	تنفيذ 50 برنامجًا مرجعيًا؛ ضاد شرح 1003 يعمل
12–10	مولّد WASM للمجموعة الرقمية؛ تحميل الوحدة في صفحة ويب؛ قياسات criterion الأولى	فيبوناتشي عربي يعمل كـ WASM في المتصفح + أول رسم أداء
14–13	خادم LSP (تشخيص + إكمال + تلميح) بـ tower-lsp؛ إضافة VS Code أولية؛ تقرير منتصف المشروع	إضافة تعمل محليًا؛ تقرير منتصف المشروع مُسلَّم

13.2 الفصل الثاني — النظام البيئي والتقييم

الأسابيع	المهام	المخرج القابل للتحقق
17–15	محرر الويب (CodeMirror 6) + طبقة BiDi بعزل الرموز؛ مجموعة اختبارات BiDi البصرية؛ المنسّق	40 حالة BiDi تجتاز؛ ضاد نسّق يعمل
20–18	الساحة: المفسر وWASM في Web Worker بحدود موارد؛ روابط المشاركة وQR؛ شارح الأخطاء (RAG + تلميحات + حارس + cache) ودمجه في LSP والساحة	تجربة «اكتب بالعربية، شغّل في المتصفح، افهم الخطأ» كاملة
22–21	بيئة التعلم: 10 دروس + المصحّح الآلي؛ المتتبّع المرئي (إن سمح الوقت)؛ نشر إضافة VS Code	مسار الدروس كامل ومنشور كصفحة ثابتة
25–23	التقييم التقني الآلي (أداء، fuzzing، LSP، دقة الإكمال، جودة الشرح سحابي/محلي)؛ دراسة المستخدمين (تجنيد، جلسات، SUS)	بيانات خام + سكربت إعادة الإنتاج

الأسابيع	المهام	المخرج القابل للتحقق
28–26	تحليل النتائج؛ كتابة التقرير النهائي؛ إصدار v1.0؛ تحضير العرض والعرض الحي الاحتياطي (فيديو)	تقرير + عرض + مستودع موسوم v1.0



الشكل 4 — المخطط الزمني على 28 أسبوعًا (الأزرق: النواة والأدوات، الذهبي: النظام البيئي، الأخضر: التقييم والتسليم).

13.3 إيقاع العمل والحوكمة

- **اجتماع أسبوعي مع المشرف** بمحضر قصير في المستودع (`docs/meetings/`).
- **سباقات (Sprints) من أسبوعين** بلوحة مهام GitHub Projects؛ كل سباق ينتهي بمخرج قابل للتشغيل.
- **قاعدة الحماية:** لا تبدأ ميزة من فئة Could قبل اكتمال كل ميزات Must في الطبقة نفسها.
- **تجميد الميزات** في الأسبوع 22؛ بعده إصلاحات وتقييم وكتابة فقط.

14 ترتيب الأولويات (MoSCoW) ونطاق MVP

Must — ضمن MVP، لا تفاوض

قواعد corpus + Tree-sitter · المفسر بالأنواع الأساسية والدوال · REPL · سياسة الأرقام وUnicode · فهرس أكواد الأخطاء (≤ 30 كودًا موثقًا) · LSP (تشخيص، إكمال، تلميح) · محرر ويب بعزل BiDi واجتياز مجموعة الاختبار · CI بالاختبارات الآلية · نواة المكتبة القياسية.

Should — مخطط له في الفصل الثاني

مولّد WASM للمجموعة الرقمية وقياس الأداء · الساحة بروابط المشاركة وQR · شارح الأخطاء بنهج RAG مع التلميحات المتدرجة والحارس والتخزين المؤقت · المنسّق · إضافة VS Code منشورة · بيئة التعلم بالمصحّح الآلي · دراسة المستخدمين بـ SUS · حدود الموارد.

Could — إن سمح الوقت

المتتبّع المرئي · مقارنة سحابي/محلي · المدقق الموسع · إجراءات الإصلاح السريع والترميز الدلالي · التوثيق المولَّد · مدير حزم مصغّر · مولّد الكود للمراجعة · WASM للنصوص والقوائم.

Won’t — ما بعد التخرج (خارطة طريق)

مترجم LLVM كامل · منقّح DAP كامل · الاستضافة الذاتية · الأصناف والوراثة الكاملة · التزامن · تطبيق جوال · مستودع حزم مركزي.

تعريف «النجاح الأدنى» صراحةً

إذا توقف المشروع في الأسبوع 20 لأي سبب، يكون المُسلَّم: لغة تعمل بمفسر وREPL، أخطاء مرقمة وموثقة، خادم LSP وإضافة VS Code، ومحرر ويب يعرض العربية المختلطة بشكل صحيح — وهو وحده يفوق ما تقدمه أي لغة عربية قائمة من حيث الأدوات. ذكر بند Won’t صراحةً أمام اللجنة يثبت وعيًا بحدود النطاق لا ضعفًا فيه.

15 التقنيات وهيكلة المستودع

15.1 حزمة التقنيات

المكوّن	التقنية	سبب الاختيار
القواعد والمحلل	`tree-sitter, tree-sitter-cli, web-tree-sitter`	تزايدي، يتعافى من الأخطاء، يعمل في Rust والمتصفح
المفسر والمترجم	`Rust, wasm-encoder, wasmtime (tests)`	أداء وسلامة؛ توليد WASM ثنائي مباشر؛ يُترجم المفسر نفسه إلى WASM
خادم اللغة	`tower-lsp, lsp-types`	تنفيذ LSP ناضج وغير متزامن في Rust
محرر الويب والساحة	`CodeMirror 6, TypeScript, Vite, Web Workers, lz-string`	دعم RTL أصلي وDecorations قابلة للبرمجة؛ بناء سريع؛ عزل التنفيذ
إضافة VS Code	`vscode-languageclient, TextMate grammar`	الطريق القياسي لعميل LSP
الذكاء الاصطناعي	`LLM API (Claude / GPT) + Ollama (Jais / AceGPT)`	طبقة تجريد واحدة تتيح التبديل والمقارنة
الاختبار والجودة	`cargo test, tree-sitter test, cargo-fuzz, criterion, Playwright, GitHub Actions`	تغطية كل الطبقات آليًا
التحليل الإحصائي	`Python (pandas, scipy), Jupyter`	تحليل قابل لإعادة الإنتاج لنتائج الدراسة

15.2 هيكلة المستودع (Monorepo)

```
dhad/
├── docs/                # محاضر الاجتماعات، التقرير، ADRs، التصميم
│   ├── adr/             # 0001-numerals.md, 0002-unicode-policy.md, ...
│   ├── spec/            # مواصفة اللغة (نحو EBNF + دلالات)
│   └── errors/          # 1003 :صفحة لكل كود.md ... (مصدر RAG)
├── grammar/             # tree-sitter-dhad: grammar.js, queries/, test/corpus/
├── crates/
│   ├── dhad-syntax/     # ربط Tree-sitter + AST + Unicode policy
│   ├── dhad-diagnostics/ # فهرس الأخطاء، التنسيق، أكواد ضاد-XXXX
│   ├── dhad-interp/     # المفسر + REPL + fuel
│   ├── dhad-wasm/       # مولّد WebAssembly
│   ├── dhad-stdlib/     # المكتبة القياسية (Rust + ضاد. ملفات)
│   ├── dhad-fmt/        # المنسّق
│   ├── dhad-lint/       # المدقق
│   ├── dhad-lsp/        # خادم اللغة
│   └── dhad-cli/        # الأمر «ضاد»: شغّل، نسّق، اختبر، شرح، وثّق
├── editors/
│   ├── vscode/          # الإضافة
│   └── web/             # محرر الويب + طبقة BiDi + الدروس + الساحة
├── ai-explainer/        # مزوّدات النماذج، cache، الحارس، RAG، بناء البرومبت
├── examples/            # 50+ برنامج مرجعي بالعربية
├── benchmarks/          # criterion + سكربت الرسوم
├── evaluation/          # أدوات الدراسة، الاستبيانات، البيانات، دفاتر التحليل
└── .github/workflows/   # CI: build, test, corpus, bidi-snapshots, bench
```

16 المخاطر وخطة التخفيف

المخاطرة	الاحتمال	الأثر	التخفيف
صعوبة توليد WASM من الصفر	مرتفع	متوسط	البدء بمجموعة رقمية صغيرة جدًا؛ استخدام wasm-encoder بدل كتابة الثنائي يدويًا؛ WASM من فئة Should لا Must
مشاكل Unicode/RTL داخل قواعد Tree-sitter	متوسط	مرتفع	نموذج أولي في الأسبوعين الأولين؛ ANTLR4 كخطة بديلة للمحلل مع إبقاء Tree-sitter للتلوين
طبقة BiDi لا تغطي كل الحالات	متوسط	مرتفع	مجموعة اختبار بصرية مبكرة؛ الاتجاه الأساسي RTL الثابت يحل معظم الحالات؛ زر تبديل يدوي كملاذ
النموذج اللغوي ينتج شروحًا غير دقيقة	متوسط	متوسط	RAG على التوثيق الرسمي + حارس جودة + عرض الرسالة الرسمية دائمًا + قياس معدل الرفض
تكلفة/انقطاع الواجهة السحابية أثناء العرض	متوسط	مرتفع	التخزين المؤقت + نموذج محلي احتياطي + فيديو عرض احتياطي
تعذّر تجنيد عدد كافٍ من المشاركين	متوسط	متوسط	التنسيق المبكر مع مدرسة/مقرر مدخل؛ خطة بديلة داخل المجموعات بـ 8–12 مشاركًا
تضخم النطاق بسبب الإضافات	مرتفع	مرتفع	MoSCoW صارم؛ تجميد الميزات في الأسبوع 22؛ أي ميزة جديدة تدخل Could افتراضيًا
كسر القواعد عند كل توسعة	مرتفع	متوسط	اختبارات golden للشجرة وcorpus في CI تكشف أي انحراف فورًا
كود الطالب يجمّد المتصفح	مرتفع	منخفض	Web Worker بمهلة إنهاء إجبارية + عدّاد وقود في المفسر
منحنى تعلم Rust	متوسط	متوسط	البدء بالمفسر البسيط؛ الاستعانة بصناديق ناضجة؛ تخصيص الأسبوعين الأولين للتدريب

17 التموضع أمام اللجنة والأسئلة المتوقعة

«المشاريع العربية السابقة إما بنت لغة تعليمية بسيطة (Alif) أو لغة أنظمة معقدة (SultanC). لم يبنِ أحد النظام البيئي الوسيط: لغة قابلة للتوسع، بأدوات مطور حقيقية، بدعم RTL أصيل، وبمساعدة ذكية للمبتدئين مقيَّدة وقابلة للقياس. هذا ما يقدمه مشروع ضاد — ويقدمه مع أرقام.»

السؤال المتوقع	الإجابة المختصرة
ما الجديد مقارنةً بألف أو SultanC؟	مصفوفة الفجوة (3.2): محلل تزايدي + BiDi + LSP هندسي + أكواد أخطاء موثقة + شارح مقيَّد + تقييم مع مستخدمين؛ لا يجمعها أي مشروع قائم.
ألا يكفي استخدام ChatGPT لشرح الأخطاء؟	الفرق في القيد والقياس: شرحنا مقيَّد بتوثيقنا الرسمي، متدرج تربويًا، يمر بحارس، ويُقاس أثره تجريبيًا — وهو جزء من المحرر لا نافذة خارجية.
لماذا Rust وليس Python؟	الأداء، ترجمة المفسر نفسه إلى WASM للعمل في المتصفح بلا خادم، ونضج صناديق tree-sitter وLSP؛ ADR-0003 يوثق المفاضلة.
ماذا لو لم تكتمل طبقة WASM؟	WASM من فئة Should؛ النجاح الأدنى (القسم 14) محدد مسبقًا ويفوق المشاريع القائمة بدونها.
كيف تضمن صحة عرض RTL؟	40+ حالة اختبار بصرية آلية في CI، واتجاه أساسي RTL ثابت مع عزل الرموز بالاعتماد على شجرة التحليل.
هل الأرقام العربية الشرقية أم الغربية؟	كلتاهما مقبولتان، تمثيل داخلي موحد، عرض حسب الإعداد — ADR-0001.
ما حجم عينة الدراسة ولماذا؟	24–30 مشاركًا بتصميم بين المجموعات يكفي لكشف أثر متوسط؛ خطة بديلة داخل المجموعات إن تعذر العدد.
ما المهارات التي يغطيها المشروع؟	مترجمات، هندسة برمجيات، تطوير ويب، تفاعل إنسان–حاسوب، ذكاء اصطناعي تطبيقي، ومنهجية بحث تجريبي.

سيناريو العرض الحي (7 دقائق)

1. فتح الساحة من رمز QR على الشاشة — أعضاء اللجنة يفتحونها على هواتفهم.
2. كتابة برنامج «الأعداد الأولية» بالعربية مع إظهار الإكمال التلقائي والتلميحات.
3. تشغيله بالمفسر ثم بـ WASM مع عرض الزمن لكل منهما.
4. إدخال خطأ متعمد (جمع نص مع عدد) ← رسالة ضاد-1003 ← طلب التلميحات المتدرجة حتى الإصلاح.
5. إظهار سطر مختلط عربي/إنجليزي في محرر عادي مقابل محرر ضاد (قبل/بعد BiDi).
6. شريحة واحدة بنتائج التقييم: الأداء، SUS، أثر الشارح.

18 المراجع

1. Alif Programming Language — لغة ألف العربية. الموقع الرسمي والمستودع المفتوح.
2. SultanC — لغة أنظمة عربية ذاتية الاستضافة؛ توثيق التمثيل الوسيط MSIR وأهداف التوليد (ELF, Mach-O, PE/COFF, WASM).
3. Zahra — مترجم عربي موحد (VM + QBE + LLVM) ومرحلة التحقق ZIR.
4. arabi-lang — لغة عربية موزعة عبر PyPI.
5. APL (2025) — استخدام نماذج اللغة الكبيرة كمترجم شبه آلي للكود العربي إلى Python (للمقارنة والتمييز).
6. Hermans, F. — Hedy: A Gradual Language for Programming Education. ICER 2020. (لغة تعليمية متعددة اللغات البشرية.)
7. Becker, B. A., et al. — Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research. ITiCSE-WGR 2019.
8. Brooke, J. — SUS: A “quick and dirty” usability scale. 1996. (المقياس المعياري لقابلية الاستخدام.)
9. Tree-sitter — An incremental parsing system for programming tools. التوثيق الرسمي.
10. Microsoft — Language Server Protocol Specification 3.17.
11. Microsoft — Debug Adapter Protocol Specification.
12. WebAssembly Community Group — WebAssembly Core Specification 2.0؛ WABT (wat2wasm)؛ صندوق wasm-encoder.
13. Unicode Consortium — UAX #9: Unicode Bidirectional Algorithm؛ UAX #15: Normalization Forms؛ UTS #39: Unicode Security Mechanisms (Confusables)؛ UAX #31: Identifier and Pattern Syntax.
14. CodeMirror 6 — توثيق دعم النص ثنائي الاتجاه والـ Decorations.
15. Guo, P. J. — Online Python Tutor: Embeddable Web-Based Program Visualization for CS Education. SIGCSE 2013.
16. Rust Project — Rust Compiler Error Index (نموذج أكواد الأخطاء الموثقة).
17. Sengupta, N., et al. — Jais and Jais-chat: Arabic-Centric Foundation and Instruction-Tuned Open Generative LLMs. 2023.
18. Wood, D., Bruner, J. S., Ross, G. — The Role of Tutoring in Problem Solving (مفهوم السقالات التعليمية). 1976.

تُستكمل الروابط وأرقام الإصدارات والصفحات في قائمة المراجع النهائية للتقرير وفق أسلوب التوثيق المعتمد في القسم (APA أو IEEE).

الملحق أ — مسودة مواصفة اللغة (نحو مبسّط بصيغة EBNF)

برنامج ::= { جملة }

جملة ::= تعريف_متغير | تعريف_ثابت | تعريف_دالة | إذا | طالما | لكل
| أرجع | توقف | استمر | استيراد | اختبار | تعبير

تعريف_متغير ::= "دع" معرّف [":" نوع] "=" تعبير

تعريف_ثابت ::= "ثابت" معرّف "=" تعبير

تعريف_دالة ::= "دالة" معرّف "(" [معرّف { فاصل معرّف }] ")" كتلة

إذا ::= "إذا" تعبير كتلة { "وإلا" "إذا" تعبير كتلة } ["وإلا" كتلة]

طالما ::= "طالما" تعبير كتلة

لكل ::= "لكل" معرّف "في" تعبير كتلة

أرجع ::= "أرجع" [تعبير]

استيراد ::= "استورد" معرّف { "." معرّف }

اختبار ::= "اختبر" نص كتلة

كتلة ::= "{" { جملة } "}"

تعبير ::= منطقي_أو

منطقي_أو ::= منطقي_و { "أو" منطقي_و }

منطقي_و ::= مساواة { "و" مساواة }

مساواة ::= مقارنة { ("==" | "!=") مقارنة }

مقارنة ::= جمع { (">" | "<" | "=>" | "=<") جمع }

جمع ::= ضرب { ("+" | "-") ضرب }

ضرب ::= أحادي { ("*" | "/" | "%") أحادي }

أحادي ::= ("ليس" | "-") أحادي | استدعاء

استدعاء ::= أولي { "(" [قائمة_معاملات] ")" | "[" تعبير "]" | "." معرّف }

أولي ::= عدد | نص | "صحيح" | "خطأ" | "عدم" | معرّف | قائمة | مدى | "(" تعبير ")"

قائمة ::= "[" [تعبير { فاصل تعبير }] "]"

مدى ::= تعبير ".." تعبير

فاصل ::= "،" | ","

عدد ::= رقم { رقم } [("." | "٫") رقم { رقم }] // رقم ∈ 0-9 ∪ ٠-٩

نص ::= '"' { محرف | "${" تعبير "}" } '"'

معرّف ::= حرف_بداية { حرف_تابع } ["؟"]

// حرف_بداية: أي حرف من فئة Letter (عربي أو لاتيني)؛ حرف_تابع: حرف أو رقم أو "_"

// تُطبَّق NFC وتُتجاهل الحركات والكشيدة وتُرفض المحارف المربكة وفق UTS #39

تعليق ::= "#" ... نهاية السطر | "##" تعليق_توثيق

الدلالات الأساسية (ملخص)

- **الأنواع:** عدد_صحيح (64-بت)، عدد_عشري (64-بت)، نص (UTF-8، الفهرسة بالمحارف لا البايتات)، منطقي، عدم، قائمة، قاموس، دالة.
- **النطاقات:** لغوية (lexical)؛ الكتل تُنشئ نطاقًا جديدًا؛ الدوال إغلاقات تلتقط البيئة.
- **الحقيقة (truthiness):** فقط `صحيح` و `خطأ` في الشروط؛ أي نوع آخر خطأ ضاد-1010 (قرار صريح لتقليل التباس المبتدئين).
- **الجمع بـ +:** عدد+عدد أو نص+نص فقط؛ غير ذلك خطأ ضاد-1003 (مع اقتراح `نص()` أو الدمج).
- **القسمة:** `/` تعطي عددًا عشريًا دائمًا؛ `//` قسمة صحيحة؛ القسمة على صفر خطأ وقت تشغيل ضاد-2001.

الملحق ب — فهرس أكواد الأخطاء الأولي

الكود	الرسالة	المساعدة المقترحة
ضاد-0001	رمز غير متوقع	عرض الرمز المتوقع في هذا الموضع من القواعد
ضاد-0002	قوس غير مغلق	الإشارة إلى موضع القوس المفتوح المقابل
ضاد-0003	نص غير مغلق	أضف علامة الاقتباس الختامية "
ضاد-0004	معرّف يحتوي محارف مربكة من نصوص مختلفة	استخدم محارف عربية فقط أو لاتينية فقط في المعرّف الواحد
ضاد-0005	كلمة مفتاحية مستخدمة كاسم متغير	اختر اسمًا آخر؛ قائمة الكلمات المحجوزة
ضاد-1001	متغير غير معرَّف	اقتراح أقرب اسم موجود (مسافة ليفنشتاين بعد التطبيع)
ضاد-1002	إعادة تعريف ثابت	الثوابت لا تتغير؛ استخدم «دع» إن أردت التغيير
ضاد-1003	لا يمكن جمع نص مع عدد	استخدم نص(العدد) أو الدمج "${...}"
ضاد-1004	عدد المعاملات لا يطابق تعريف الدالة	عرض توقيع الدالة المتوقع
ضاد-1005	استدعاء قيمة ليست دالة	عرض نوع القيمة الفعلي
ضاد-1006	«أرجع» خارج دالة	انقل الجملة داخل دالة أو احذفها
ضاد-1007	«توقف»/«استمر» خارج حلقة	تُستخدمان داخل طالما أو لكل فقط
ضاد-1008	مقارنة بين نوعين غير متوافقين	حوّل أحد الطرفين
ضاد-1009	وحدة غير موجودة عند الاستيراد	قائمة الوحدات المتاحة
ضاد-1010	الشرط ليس قيمة منطقية	استخدم مقارنة صريحة مثل س != ٠
ضاد-2001	قسمة على صفر	تحقق من المقسوم عليه قبل القسمة
ضاد-2002	فهرس خارج حدود القائمة	الفهارس تبدأ من ٠ وتنتهي عند طول-١
ضاد-2003	مفتاح غير موجود في القاموس	تحقق بـ «في» قبل الوصول
ضاد-2004	تجاوز عمق الاستدعاء الذاتي	تحقق من شرط التوقف في الدالة
ضاد-2005	تجاوز المهلة الزمنية للتنفيذ	ربما توجد حلقة لا تنتهي؛ تحقق من شرط الحلقة
ضاد-3001	تحذير: متغير معرَّف وغير مستخدم	احذفه أو استخدمه
ضاد-3002	تحذير: معرّفان لا يختلفان إلا بالهمزة أو التاء المربوطة	وحّد التهجئة لتجنب الالتباس
ضاد-3003	تحذير: تشكيل داخل معرّف (تم تجاهله)	احذف الحركات من أسماء المتغيرات

الملحق ج — قالب سجل القرار المعماري (ADR)

ADR-0001: التعامل مع الأرقام العربية الشرقية والغربية

الحالة: مقبول — التاريخ: ____

السياق: يختلف مستخدمو المشرق والمغرب في الأرقام المعتادة؛ المترجم يحتاج تمثيلًا واحدًا.

القرار: يقبل المحلل الصنفين ويوحّدهما إلى الغربي داخليًا؛ العرض خاضع لإعداد المستخدم.

البدائل المرفوضة: (١) الشرقية فقط — يُقصي المغرب العربي. (٢) الغربية فقط — يناقض هدف العربية الأصيلة.

العواقب: + بساطة المترجم + مرونة العرض - الحاجة إلى اختبارات للصنفين في كل مكان.

الملحق د — استبيان SUS (الترجمة العربية المعتمدة في الدراسة)

يُجاب عن كل عبارة بمقياس من 1 (لا أوافق بشدة) إلى 5 (أوافق بشدة). تُحسب الدرجة وفق معادلة Brooke القياسية (0–100).

1. أعتقد أنني سأرغب في استخدام هذه المنصة بشكل متكرر.
2. وجدت المنصة معقدة أكثر مما ينبغي.
3. اعتقدت أن المنصة سهلة الاستخدام.
4. أعتقد أنني سأحتاج إلى دعم شخص تقني لأتمكن من استخدام هذه المنصة.
5. وجدت أن الوظائف المختلفة في هذه المنصة متكاملة بشكل جيد.
6. اعتقدت أن هناك الكثير من التناقض في هذه المنصة.
7. أتصور أن معظم الناس سيتعلمون استخدام هذه المنصة بسرعة كبيرة.
8. وجدت المنصة مرهقة جدًا في الاستخدام.
9. شعرت بثقة كبيرة أثناء استخدام المنصة.
10. احتجت إلى تعلم الكثير من الأشياء قبل أن أتمكن من البدء باستخدام هذه المنصة.

الملحق هـ — قاموس المصطلحات

المصطلح	التعريف
محلل تزايدي (Incremental Parser)	محلل يحدّث شجرة التحليل عند كل تعديل بدل إعادة بنائها، مما يتيح الاستجابة الفورية في المحرر.
شجرة التحليل (Syntax Tree)	تمثيل هرمي لبنية البرنامج يُستهلك من المفسر والمترجم والأدوات.
مفسر Tree-walking	مفسر ينفذ البرنامج بالمشي على شجرته مباشرة دون توليد كود وسيط.
WebAssembly (WASM)	صيغة ثنائية محمولة وآمنة تعمل في المتصفحات وخارجها بأداء قريب من الأصلي.

المصطلح	التعريف
LSP	بروتوكول خادم اللغة؛ معيار يتيح لأي محرر الحصول على الإكمال والتشخيص من خادم واحد.
BiDi / UAX #9	خوارزمية Unicode لعرض النص ثنائي الاتجاه (عربي مع لاتيني وأرقام).
عزل اتجاهي (Isolation)	منع تسرب اتجاه مقطع نصي إلى جيرانه المحايدين.
RAG	التوليد المعزز بالاسترجاع: تزويد النموذج اللغوي بمصدر موثوق ليعتمد عليه بدل معرفته العامة.
حارس الجودة (Guardrail)	فحوص آلية على مخرجات النموذج قبل عرضها للمستخدم.
السقالات التعليمية (Scaffolding)	تقديم المساعدة تدريجيًا بقدر حاجة المتعلم ثم سحبها.
SUS	مقياس قابلية الاستخدام المعياري من 10 عبارات بدرجة 0–100 (المتوسط العالمي 68).
Fuzzing	اختبار المتانة بمدخلات عشوائية أو مشوّهة لاكتشاف الانهيارات.
ADR	سجل قرار معماري: وثيقة قصيرة تحفظ سياق قرار تصميمي وبدائله وعواقبه.
MoSCoW	أسلوب ترتيب أولويات: Must / Should / Could / Won’t.

— نهاية الوثيقة — منصة ضاد · وثيقة مشروع التخرج الشاملة · الإصدار 2.0